



Programmation Python

Filière LSI1



Chapitre 2:

les structures de données en Python

2020-2021 - SEMESTRE 2

nassima.medimegh@fsm.rnu.tn



1. Les séquences

- Une séquence est un conteneur **ordonné** d'éléments indexés par des entiers indiquant leurs positions dans le conteneur
- Les indices commencent par **0**
- Soit **S** une séquence:
 - **len(S)**: retourne le nombre d'éléments dans la séquence **S**
 - **S[i]**: retourne $(i+1)$ ème élément de **S** avec $0 \leq i < \text{len}(S)$.
 - **S [:]**: tous les éléments de la séquence
 - **S [: a]** : une sous séquence d'éléments de début jusqu'à l'élément d'indice **a** exclu
 - **S [a :]**: une sous séquence d'éléments de l'indice **a** inclus jusqu'à la fin
 - **S [a : b]** : sous séquence d'éléments allant de l'indice **a** inclus jusqu'à l'élément d'indice **b** exclu



1. Les séquences

- **S [a: b:c]** : une sous séquence d'éléments de l'indice **a** jusqu'à l'indice **b** exclu par pas d'incrémenté égal à **c**.
- **S [: :c]** : une sous séquence d'éléments de l'indice **0** jusqu'à la fin avec un pas d'incrémenté égal à **c**
- **S [a: :c]** : une sous séquence d'éléments de l'indice **a** jusqu'à la fin avec un pas d'incrémenté égal à **c**
- **S [: b :c]** : une sous séquence d'éléments de l'indice **0** jusqu'à l'indice **b** avec un pas d'incrémenté égal à **c**
- **S [: :-1]** : l'image miroir de la séquence
- **S [-k : :]** : les **k** derniers éléments



2. Les chaînes de caractères

- Une chaîne de caractère est une suite de caractères entourée par deux **apostrophes** ' ou deux **guillemets** "
- C'est une Structure **indexée**, **ordonnée** et **non modifiable**.
- Le type caractère en Python n'existe pas comme dans certain langage, un caractère en Python est une chaîne d'un caractère.

```
chaîne = 'Une chaîne'
```

immutable

U n e c h a î n e

0 1 2 3 len()-1

-len() -4 -3 -2 -1

chaîne[2] => 'e'



2. Les chaines de caractères

- La séquence `\n` dans une chaîne provoque un saut à la ligne.
- La séquence `\'` permet d'insérer une apostrophe dans une chaîne délimitée par des apostrophes
- Rappelons encore ici que la casse est significative dans les noms de variables (Il faut respecter scrupuleusement le choix initial de majuscules ou minuscules).

```
>>> CH, ch= "BONJOUR", 'bonjour'
>>> CH, ch
('BONJOUR', 'bonjour')
>>> type(ch)
<class 'str'>
```

```
>>> CH[2]= 'b'
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    CH[2]= 'b'
TypeError: 'str' object does not support item
assignment
```



2. Les chaînes de caractères

Fonctions et méthodes de chaînes :

- L'opérateur **+**: concaténation de chaînes.
- L'opérateur *****: duplication de chaînes

```
>>> n= 'abc' + 'def'
>>> m= 'Bonjour !' *3
>>> print(" n= ", n, " \n m= ",m)
n= abcdef
m= Bonjour ! Bonjour ! Bonjour !
```

- Pour extraire un caractère d'une chaîne, il suffit d'indiquer son indice *entre crochets*

```
>>> M="Python"
>>> print (len(M), M[0], M[5])
6 P n
```



2. Les chaînes de caractères

Fonctions et méthodes de chaînes :

- *Slicing* (découpage en tranches)

```
>>> M="Python"
>>> print (len(M), M[0], M[5])
6 P n
>>> M[0:2]
'Py'
>>> M[0:-3]
'Pyt'
>>> M[:3] # les 3 premiers caractères
'Pyt'
>>> M[3:] # tout sauf les 3 premiers caractères
'hon'
```



2. Les chaînes de caractères

Parcours d'une chaîne de caractère

```
>>> nom = 'Python'
>>> for caract in range(len(nom)) :
    print (nom[caract] + ' *')
```

```
P *
y *
t *
h *
o *
n *
```

```
>>> nom = 'Python'
>>> for caract in nom :
    print (caract + ' *')
```

```
P *
y *
t *
h *
o *
n *
```




2. Les chaines de caractères

Quelques fonctions prédéfinies

Méthodes	Rôles
lower() et upper()	renvoient un texte en minuscule et en majuscule respectivement.
isalpha()	Retourne vrai si la chaîne a au moins 1 caractère et que tous les caractères sont alphabétiques et faux sinon.
isalnum()	Retourne true si la chaîne a au moins 1 caractère et que tous les caractères sont alphanumériques et false sinon.
isspace()	Renvoie true si la chaîne ne contient que des caractères d'espacement et false sinon.
max(str)	Renvoie le caractère alphabétique maximal de la chaîne str.
find('caractère')	Retourner l'index de l'élément 'caractère' et -1 s'il n'existe pas



2. Les chaines de caractères

Quelques fonctions prédéfinies

Méthodes	Rôles
startswith()	vérifie si une chaîne de caractères commence par une autre chaîne de caractères
index(c)	retrouve l'index de la première occurrence du caractère c dans la chaîne
format()	permet une meilleure organisation de l'affichage des variables
strip()	enlève les espaces situés sur les bords de la chaîne de caractère
replace()	substituer une chaîne de caractères par une autre
count()	compte le nombre d'occurrences d'une chaîne de caractères passée en argument



2. Les chaines de caractères

Quelques exemples

```
>>> x="python"
>>> x.upper()
'PYTHON'
>>> x.lower()
'python'
>>> x.capitalize()
'Python'
>>> x.find("y")
1
>>> x.find("on")
4
>>> x.count("o")
1
>>> x.startswith("pyt")
True
>>> x.replace("on", " prog")
'pyth prog'
>>> y="FSM"
>>> print("{} a {}".format(x,y))
python a FSM
```



2. Les chaines de caractères

Exercice

- Ecrivez un script qui détermine si une chaîne de caractères donnée est un palindrome (c'est-à-dire une chaîne qui peut se lire indifféremment dans les deux sens), comme par exemple << radar >>

```
>>> i='radar'  
>>> invers=i[::-1]
```

python 3. Les listes

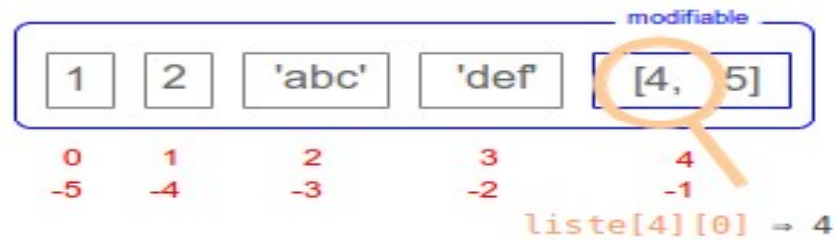
- Une liste en Python est un type de données **ordonnée** et **modifiable**.
- les listes sont écrites entre crochets [].
- Les éléments qui constituent une liste peuvent être de **types variés**, voire même des listes.
- Une liste peut être **vide**.
- Les éléments d'une liste peuvent contenir des expressions complexes, faisant appel à des variables ou des fonctions :

$a = [(2 + 3) * b, c * 2]$

python 3. Les listes

Création d'une liste

```
liste = [1, 2, 'abc', 'def', [4,5]]
```



```
>>> #Création d'une l i s t e
>>> langage = [ "Python" , "Java" , "PHP" ]
>>> langage
['Python', 'Java', 'PHP']
>>> liste_vide=[]
>>> liste_vide
[]
```

python 3. Les listes

Création d'une liste

- Il est possible de créer une liste de nombres à l'aide de la fonction `range()`.

<code>range(n)</code>	génère une liste renfermant les nombres : 0, 1, 2, ..., n-1; si $n \leq 0$, cela génère une liste vide [].
<code>range(m, n)</code>	génère une liste renfermant les nombres : m, m+1, m+2, ..., n-1; si $m \geq n$, cela génère une liste vide [].
<code>range(m, n, p)</code>	génère une liste renfermant les nombres : m, m+p, m+2p, ..., m+kp, où k est le plus grand entier tel que : m+kp < n si $p > 0$, m+kp > n si $p < 0$.



3. Les listes

Création d'une liste

```
>>> L1=list(range(12))
>>> L1
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
>>> L2=list(range(0))
>>> L2
[]
>>> L3=list(range(3,7))
>>> L3
[3, 4, 5, 6]
>>> L4=list(range(6, 12, 2))
>>> L4
[6, 8, 10]
```




3. Les listes

Création d'une liste par compréhension

- On peut aussi définir une liste par compréhension avec la syntaxe : `[ fonction(x) for x in liste if test(x) ]`

```
>>> L=[2*x for x in range(10)]
>>> L
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
>>> L1=[x for x in range(10) if x>5]
>>> L1
[6, 7, 8, 9]
>>> L2=[x for x in range(10) if x>5 and x<8]
>>> L2
[6, 7]
>>> L3=[2*x for x in [x+1 for x in range(10)]]
>>> L3
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
>>> L4=[i for i in range(10) if i % 2 == 0]
>>> L4
[0, 2, 4, 6, 8]
```

python 3. Les listes

Exercice

- Créer une liste par compréhension permettant d'afficher les entiers cubiques compris entre 100 et 999.
- Un entier est dit cubique s'il est égal à la somme des cubes de ses chiffres.
- Exemple : $153 = 1^3 + 5^3 + 3^3$

```
>>> prime=[i for i in range(100,999) if (i//100)**3 +(i%10)**3+((i//10)%10)**3==i ]  
>>> prime  
[153, 370, 371, 407]
```

python 3. Les listes

- **Exercice 2**
- Obtenir tous les nombres premiers inférieur ou égaux à 50 en utilisant les listes

python 3. Les listes

Accès aux éléments d'une liste

- Vous accédez aux éléments d'une liste Python, en vous référant au numéro d'index :

```
>>> langage = [ "Python" , "Java" , "PHP" ]  
>>> langage[1]  
'Java'
```

python 3. Les listes

Changer la valeur d'un élément de la liste

- Pour modifier la valeur d'un élément spécifique, reportez-vous au numéro d'index :

```
>>> langage = [ "Python" , "Java" , "PHP" ]
>>> langage[1]
'Java'
>>> langage[0]= "Oracle"
>>> langage
['Oracle', 'Java', 'PHP']
```

python 3. Les listes

Parcourir les éléments d'une liste Python

- Vous pouvez parcourir les éléments d'une liste en Python, en utilisant une boucle for :

```
>>> myList = [ "Formation" , "Python" , "3"]
>>> for x in myList :
    # Afficher tous les éléments de la l i s t e un par un
    print (x)
```

```
Formation
Python
3
.
```

python 3. Les listes

Concaténation de listes (opérateur +)

```
>>> myList = [ "Formation" , "Python" , "3"]
>>> langage = [ "Java" , "PHP" ]
>>> Liste = myList + langage
>>> Liste
['Formation', 'Python', '3', 'Java', 'PHP']
|
```

python 3. Les listes

Répétition de listes (opérateur *)

```
>>> position = [0, 1, 2, 3, 4, 5]
>>> position = position * 2
>>> position
[0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5]
>>> vecteur_nul = [0] * 3
>>> vecteur_nul
[0, 0, 0]
```

```
>>> print (["_"] * 5)
['_ ', '_ ', '_ ', '_ ', '_ ']
```

```
>>> s = ["plume", 3, True] * 3
>>> s
['plume', 3, True, 'plume', 3, True, 'plume', 3, True]
```


python 3. Les listes

Méthodes associées aux listes

.append()

- ajoute un élément à la fin d'une liste :

```
>>> a = [1, 2, 3]
>>> a.append(5)
>>> a
4 [1, 2, 3, 5]
```

python 3. Les listes

Méthodes associées aux listes

.extend(L)

- ajoute les éléments de la liste **L** à la fin d'une liste

```
>>> L1=['a',7,'bon',87,4,6,7]
>>> L1.extend('r')
>>> L1
['a', 7, 'bon', 87, 4, 6, 7, 'r']
>>> L1.extend("bonjour")
>>> L1
['a', 7, 'bon', 87, 4, 6, 7, 'r', 'b', 'o', 'n', 'j', 'o', 'u', 'r']
>>> L1.extend([5,50])
>>> L1
['a', 7, 'bon', 87, 4, 6, 7, 'r', 'b', 'o', 'n', 'j', 'o', 'u', 'r', 5, 50]
```

python 3. Les listes

Méthodes associées aux listes

.insert()

- insère un objet dans une liste avec un indice déterminé :

```
>>> L=[1,8,7]
>>> L.insert(1,45)
>>> L
[1, 45, 8, 7]
>>> L.insert(2,[1,2,3])
>>> L
[1, 45, [1, 2, 3], 8, 7]
```

python 3. Les listes

Méthodes associées aux listes

del

- supprime des éléments d'une liste à un indice déterminé :

```
>>> a = [1, 2, 3, 4, 5]
```

```
>>> del a [1]
```

```
>>> a
```

```
[1, 3, 4, 5]
```

```
>>> del a [2:4]
```

```
>>> a
```

```
[1, 3]
```

python 3. Les listes

Méthodes associées aux listes

.remove()

- supprime un élément d'une liste à partir de sa valeur :

```
>>> a = [1, 2, 3]
>>> a.remove(3)
>>> a
[1, 2]
```

python 3. Les listes

Méthodes associées aux listes

.sort()

- trie une liste :
 - Par ordre croissant si les éléments sont des nombres.
 - Par ordre alphabétique si les éléments sont des caractères alphabétiques.

```
>>> a = [3, 1, 2]
>>> a.sort ()
>>> a
[1, 2, 3]
```

python 3. Les listes

Méthodes associées aux listes

.reverse()

- inverse l'ordre des éléments d'une liste

```
1 >>> a = [3, 1, 2]
2 >>> a.reverse ()
3 >>> a
4 [2, 1, 3]
```

python 3. Les listes

Méthodes associées aux listes

.count()

- compte le nombre d'éléments (passés en argument) dans une liste :

```
>>> a = [1, 2, 4, 3, 1, 1]
>>> a.count(1)
3
>>> a.count(4)
1
>>> a.count(23)
0
```




3. Les listes

Copie d'une liste

- L'affectation d'une liste à une variable ne crée pas une copie de la liste. Voici un exemple de programme qui illustre ce phénomène :

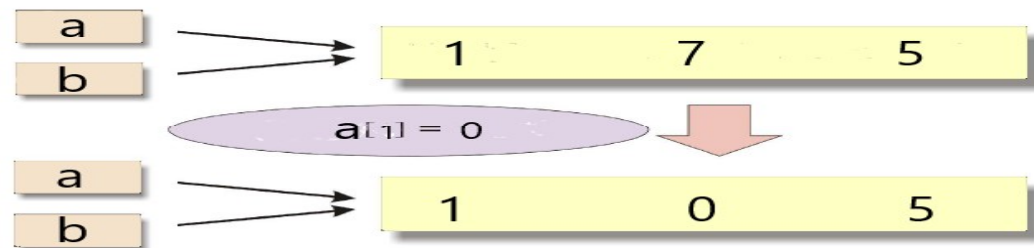
```
>>> a = [1, 7, 5]
```

```
>>> b = a
```

```
>>> a[1] = 0
```

```
>>> b
```

```
[1, 0, 5]
```



- On voit que la modification de la liste a affecté aussi la liste b
- En fait, les noms **a** et **b** désignent tous deux un seul et même objet en mémoire. Pour décrire cette situation, les informaticiens diront que le nom **b** est un *alias* du nom **a**.



3. Les listes

Copie d'une liste

- Pour faire une nouvelle copie de la liste, il est nécessaire d'utiliser une expression comme ci-dessous

```
>>> a = [1, 7, 5]
```

```
>>> b = a[:] # b = a + [],
```

```
ou b = list(a) ou b = [ x for x in a ]
```

```
>>> a[1]= 0
```

```
>>> b
```

```
[1, 7, 5]
```

```
>>> a=[1,7,5]
>>> b=a[:]
>>> c=a+[]
>>> d=list(a)
>>> e=[x for x in a]
>>> a
[1, 7, 5]
>>> b
[1, 7, 5]
>>> c
[1, 7, 5]
>>> d
[1, 7, 5]
>>> e
[1, 7, 5]
>>> a[1]=0
>>> b
[1, 7, 5]
>>> a
[1, 0, 5]
>>> c
[1, 7, 5]
... |
```

python 3. Les listes

Transformer une liste en une chaîne de caractères **.join()**

```
>>> I=["Bonjour", "Le TP", "Python", "aura lieu" ,"cette semaine"]
>>> S=" ".join(I)
>>> S
'Bonjour Le TP Python aura lieu cette semaine'
>>> I
['Bonjour', 'Le TP', 'Python', 'aura lieu', 'cette semaine']
```

python 3. Les listes

Transformer une chaîne de caractères en une liste

.split()

```
>>> S
'Bonjour Le TP Python aura lieu cette semaine'
>>> Liste=S.split()
>>> Liste
['Bonjour', 'Le', 'TP', 'Python', 'aura', 'lieu', 'cette', 'semaine']
|
```



4. Les tuples

- Un tuple est une collection **ordonnée** et **non modifiable** (n-uplets en mathématiques) qui est assez semblable a une liste.
- Il est écrit avec des parenthèses ().
- Les éléments d'un tuple ont un ordre défini, tout comme ceux d'une liste. Les indices de tuples débutent à zéro, tout comme ceux d'une liste, le premier élément d'un tuple non vide est toujours t[0].

```
>>> myTuple=("Python", "Linux", "JAVA")
>>> myTuple
('Python', 'Linux', 'JAVA')
```

python 4. Les tuples

- Les **opérations** que l'on peut effectuer sur des tuples sont syntaxiquement similaires à celles que l'on effectue sur les listes.
- Ils n'ont pas des méthodes pour ajouter, modifier ou supprimer ses éléments.
- Deux méthodes intégrées que vous pouvez utiliser sur des tuples:
 - **count ()** Retourne le nombre de fois qu'une valeur spécifiée apparaît dans un tuple.
 - **index ()** Recherche dans le tuple une valeur spécifiée et renvoie la position de l'endroit où il a été trouvé.

python 4. Les tuples

Mais à quoi servent donc les tuples ?

- Les tuples sont plus rapides que les listes. Si vous définissez un ensemble constant de valeurs et que tout ce que vous allez faire est le parcourir, utilisez un tuple au lieu d'une liste.
- Votre code est plus sûr si vous «protégez en écriture» les données qui n'ont pas besoin d'être modifiées.

python 4. Les tuples

Exercice 2 :

- Calcul des racines réelles du trinôme : $ax^2 + bx + c = 0$
- Juste après l'en-tête du fichier, insérez l'instruction : **from math import sqrt**
- Écrire un programme prenant trois paramètres a b et c, et qui affiche un tuple qui contient les racines flottantes de l'équation du second degré $ax^2+bx+c=0$. Le tuple sera vide s'il n'y a pas de racine, ou contiendra une ou deux valeurs flottantes.
- **Delta**= (b^2-4ac)
- $(-b-\text{racineDeDelta})/(2*a)$
- $(-b+\text{racineDeDelta})/(2*a)$
- $-b/(2*a)$



5. Les dictionnaires

- Structure de donnée **non ordonnée** et **mutable** permettant de stocker une liste de paires **clé:valeur**
- La syntaxe pour associer une clé et une valeur est donc la suivante: clé:valeur (avec deux-points **:**), pour déclarer plusieurs clés il suffit de les séparer par des virgules **,**

Syntaxe :

```
dic = {key1: valeur1, key2: valeur2, key3: valeur3, ...}
```

```
>>> dico={ 'langage':'python','system':'windows','number':1}  
>>> dico  
{'langage': 'python', 'system': 'windows', 'number': 1}  
>>> dico['langage']
```



5. Les dictionnaires

- Les "keys" et "values" ne sont pas obligatoirement des chaînes de caractères, ils peuvent parfaitement être d'autres types comme "int".
- De plus, il est aussi possible d'associer plusieurs valeurs à un même mot clé.

```
>>> dico={'langage':('Python', 'JAVA', 'C'),'Systeme': 'windows','number':(7,10)}  
>>> dico  
{'langage': ('Python', 'JAVA', 'C'), 'Systeme': 'windows', 'number': (7, 10)}
```

python Les dictionnaires

Opérations sur les dictionnaires

```
>>> dico
{'langage': 'python', 'system': 'windows', 'number': 1}
>>> dico['langage']
'python'
>>> dico['computer']='keyboard' #ajout d'un element au dictionnaire
>>> dico
{'langage': 'python', 'system': 'windows', 'number': 1, 'computer': 'keyboard'}

>>> dico
{'langage': 'python', 'system': 'windows', 'number': 1}
>>> dico.pop( 'langage') # supprimer un element
'python'
>>> dico
{'system': 'windows', 'number': 1}
>>> dico.popitem()#Suppression du dernier élément
('number', 1)
>>> dico
{'system': 'windows'}
```



Les dictionnaires

Opérations sur les dictionnaires

keys() renvoie la liste des clés utilisées dans le dictionnaire

values() renvoie la liste des valeurs mémorisées dans le dictionnaire

items() extrait du dictionnaire une liste équivalente de tuples

Clear() Vider un dictionnaire

in /not in: Vérifier l'appartenance d'une clé dans le dictionnaire.

get : extrait la valeur associée à une clef

python Les dictionnaires

Opérations sur les dictionnaires

```
>>> dico
{'langage': 'python', 'system': 'windows', 'number': 1}
>>> # la liste des clés
>>> dico.keys()
dict_keys(['langage', 'system', 'number'])
>>> # la liste des valeurs
>>> dico.values()
dict_values(['python', 'windows', 1])
>>> # les clés et les valeurs en même temps
>>> dico.items()
dict_items([('langage', 'python'), ('system', 'windows'), ('number', 1)])
```

```
>>> dico={'langage':'python','system':'windows','number':1}
>>> 1 in dico
False
>>> 'JAVA' in dico
False
>>> dico.clear()
>>> dico
{}
.
```



Les ensembles

- Un ensemble est une collection d'éléments distincts, **non ordonnés** et **non dupliqués**. On peut créer un ensemble en listant tous ses éléments, séparés par des **virgules**, et en délimitant le tout avec des **accolades**.
- On ne peut par contre pas accéder aux éléments d'un ensemble à partir d'un indice, puisqu'il s'agit d'une collection non ordonnée d'éléments, et on ne peut donc parcourir les éléments d'un ensemble qu'avec la boucle for.



Les ensembles

```
# Définition
numbers = {42, -2, 0, 7, 11}
# seconde façon de définir un set avec doubles ( )
set1 = set(['b','c','d','c']) # {'c', 'b', 'd'}
print(numbers)                # {0, 42, 11, -2, 7}
print(len(numbers))           # 4
print(60 in numbers)          # False
print(type(numbers))          # < class ' set '>
for element in numbers :
    print ( element )
```



Les ensembles

Opérations sur les ensembles

- **add()** : ajoute un élément à l'ensemble
- **clear()** : supprime tous les éléments de l'ensemble
- **copy()** : retourne une copie de l'ensemble
- **difference ()** : retourne un ensemble contenant la différence entre deux ensembles ou plus.
- **discard()** : supprimer l'élément spécifié
- **intersection()** : retourne un ensemble, qui est l'intersection de deux autres ensembles.
- **pop()** : supprime un élément de l'ensemble
- **remove()** : supprime l'élément spécifié
- **union()** : retourne un ensemble contenant l'union des ensembles
- **update()** : met à jour l'ensemble avec l'union de cet ensemble et d'autres

python Les ensembles

Opérations sur les ensembles

```
S = {1, 2, 3, 4}
```

```
S.add(5)          # ajouter 1 seul élément → {1, 2, 3, 4, 5}
```

```
S.remove(5)       # détruire 1 seul élément → {1, 2, 3, 4}
```

```
S.discard(4)      # remove, mais pas erreur si l'élément est absent → {1, 2, 3}
```

```
S.clear()         # détruire tous les éléments → { }
```

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5}
```

```
print( A - B )    # différence → {1, 2}
```

```
print( A & B )    # intersection → {3, 4}
```

```
print( A | B )    # union → {1, 2, 3, 4, 5}
```